The C++ interview question bank

For developers with 3+ years of professional experience. Practice explaining real failure modes, reviewing small programs and defending design decisions.

96	12	96
primary questions	topic sets	follow-up probes

CODE REVIEW / R02

Will this pointer survive?

```
std::vector<int> v{4, 9};
auto* p = &v.front();
v.push_back(16);
use(*p);
```

Try the code-review round ›

How to rehearse

Answer aloud before opening the cues. Name the guarantee, illustrate it with a failure case and say how you would test the fix. Three timed mock interviews are included.

Browse questions and answer cues ›

Aman Choudhari

10 November 2026

INTERVIEW GUIDE

Contents

Jump to the questions for each topic, or open its answer cues directly.

	TOPIC	QUESTIONS	CUES
L	Lifetime, references and linkage	4	17
M	Ownership and move semantics	5	18
P	Polymorphism and object design	6	19
S	Containers and algorithms	7	20
G	Templates and modern C++	8	21
E	Exceptions and API contracts	9	22
T	Concurrency and the memory model	10	23
D	Debugging, performance and builds	11	24
R	Code review at the whiteboard	12	25
A	Live coding and algorithms	14	26
X	CAD, geometry and plug-ins	15	27
W	Delivery and engineering judgment	16	28
	Three timed mock interviews		3
	Technical references		29

Language baseline

Most prompts work in C++17. Questions about concepts, ranges, std::span, std::jthread, std::erase_if and atomic<shared_ptr<T>> use C++20. std::expected is C++23. If your production code targets an earlier version, state the version and describe a compatible implementation.

Reading the answer cues

The cues identify guarantees, trade-offs and observable checks. Several designs can satisfy the same contract. For experience questions, use your own facts, measurements and decisions.

PRACTICE ROUNDS

Three mock interviews

Select the question ID for the full prompt. The timings include the follow-ups.

General C++ screening — 45 min

5 min	W01 ›	Explain a defect you took from report to regression test.
10 min	M03 ›	Reason about return by value and copy elision.
20 min	A01 ›	Code first occurrence in a sorted vector and test its bounds.
10 min	R04 ›	Review the erase loop and justify a one-pass repair.

Production C++ and debugging — 50 min

5 min	W02 ›	Ground an automation speedup in measured evidence.
12 min	D01 ›	Investigate a crash tied to a vector growing.
25 min	A06 ›	Design the queue and implement its blocking and closing rules.
8 min	T03 ›	Walk a consumer through a wakeup and a shutdown.

CAD or geometry developer — 60 min

8 min	X05 ›	Trace an assembly transform using a minimal test.
12 min	P01 ›	Define polymorphic ownership across a plug-in interface.
25 min	X03 ›	Implement a hull and choose a collinearity policy.
15 min	X08 ›	Move a solver off the UI thread and reject stale results.

Score each response from 0 to 3

0	Could not identify the relevant rule or approach.
1	Named a plausible approach but missed a failure mode or boundary.
2	Gave a correct contract, a sound approach and one concrete test.
3	Also handled the follow-up and explained a trade-off or measurable cost.

QUESTION SET L

Lifetime, references and linkage

Trace initialization, value categories, lifetime and translation-unit boundaries.

L01 Answer cues 17 ›

A parser returns a std::string_view for a field it found. Which objects must outlive the caller's use of that view?

Follow-up What if the backing string grows or the input is a temporary?

L02 Answer cues 17 ›

An overload set has f(T&), f(const T&) and f(T&&) for a concrete class T. Which overload receives a mutable variable, a const variable and a temporary?

Follow-up Inside f(T&& x), what category is the expression x?

L03 Answer cues 17 ›

An object holds a pointer into its own std::string. Its tests pass until someone copies it. Which invariant is broken?

Follow-up Would default move construction reliably preserve the invariant?

L04 Answer cues 17 ›

A header defines a non-inline function and is included by two .cpp files. Why can compilation pass while linking fails?

Follow-up How would the answer change for a header-defined inline function?

L05 Answer cues 17 ›

A public header needs a compile-time gravity constant shared across translation units. Compare namespace-scope const, static and inline constexpr in C++17.

Follow-up When is the address of the object significant?

L06 Answer cues 17 ›

A timer computes 9.8 * ((t * t) / 2) with int t. Why is its first nonzero reading late?

Follow-up What second failure appears at large t?

L07 Answer cues 17 ›

Two threads call a function with a dynamically initialized function-local static cache. What does C++11 synchronize?

Follow-up What about a later write to the cache?

L08 Answer cues 17 ›

A decoder reads a 32-bit field by casting a byte pointer and dereferencing it. What must a code review investigate?

Follow-up How would you read a little-endian value portably?

QUESTION SET M

Ownership and move semantics

Make every owner, observer and transfer in the example explicit.

M01 Answer cues 18 ›

Design a move-only wrapper around an SDK handle with an invalid sentinel and a required release call.

Follow-up What happens on self-move assignment and on a failed acquisition?

M02 Answer cues 18 ›

What actually happens when std::move is applied to a const std::string before passing it by value?

Follow-up When can an operation with std::move still be expensive?

M03 Answer cues 18 ›

A factory returns a large local mesh by value. Should it return std::move(mesh)?

Follow-up Distinguish a named local from returning Mesh{...} in C++17.

M04 Answer cues 18 ›

Which parameter types would you choose for observing, taking ownership of, or sharing ownership of a document?

Follow-up How do you express an optional observer?

M05 Answer cues 18 ›

A vector grows while relocating elements whose move constructor may throw. Why does the copy constructor matter?

Follow-up Which guarantee becomes harder when copying is unavailable?

M06 Answer cues 18 ›

Two shared_ptr values are built independently from the same raw pointer. Both report use_count() == 1. Diagnose it.

Follow-up How should an object safely publish a shared pointer to itself?

M07 Answer cues 18 ›

A view owns its controller through shared_ptr, and the controller owns the view. How would you make shutdown deterministic?

Follow-up How does a callback safely use a weak_ptr?

M08 Answer cues 18 ›

An RAII class declares a destructor purely for logging. A refactor now copies large members. What changed?

Follow-up How would you preserve the intended copy and move policy?

QUESTION SET P

Polymorphism and object design

Connect interface contracts to dispatch, destruction and object representation.

P01 Answer cues 19 ›

A plug-in factory returns unique_ptr<Base> that actually owns a Derived. What must the base guarantee with the default deleter?

Follow-up When is a protected nonvirtual destructor appropriate?

P02 Answer cues 19 ›

A renderer takes Shape by value. A derived circle reaches it with no circle-specific behavior. Where did the information go?

Follow-up How would a container keep several dynamic shape types?

P03 Answer cues 19 ›

A base constructor calls a virtual hook that a derived class overrides. Which implementation is selected?

Follow-up How would you run derived setup after full construction?

P04 Answer cues 19 ›

An override silently stops overriding after a const qualifier changes. How can tooling and the interface catch it?

Follow-up Does a matching name also preserve every base overload?

P05 Answer cues 19 ›

A drawing plug-in needs interchangeable export strategies, but an exported entity is not itself an exporter. How would you model the relationship?

Follow-up Who owns the strategy and how is it replaced during a job?

P06 Answer cues 19 ›

A library introduces virtual clone() to copy heterogeneous shapes. How would you specify its return type and ownership?

Follow-up Can an override returning unique_ptr<Circle> replace unique_ptr<Shape>?

P07 Answer cues 19 ›

A hierarchy is used only in one hot loop. How would you decide between virtual dispatch, std::variant and a templated loop?

Follow-up Which inputs make the benchmark representative?

P08 Answer cues 19 ›

A derived type rejects inputs the base contract promises to accept. What will fail even though every override compiles?

Follow-up Give a test that can be reused for each implementation.

QUESTION SET S

Containers and algorithms

State iterator validity and cost for the exact container operation.

S01 Answer cues 20 ›

A vector stores mesh vertices. A caller caches a vertex pointer and later appends vertices. When is that pointer valid?

Follow-up Does reserve() protect it after middle erasure?

S02 Answer cues 20 ›

Choose between vector, map and unordered_map for an ID index that must occasionally export sorted IDs.

Follow-up What if lookups dominate and keys arrive in adversarial order?

S03 Answer cues 20 ›

A loop erases matching items from a vector. How do you preserve validity and avoid quadratic shifting?

Follow-up How would you retain the relative order of survivors in C++17?

S04 Answer cues 20 ›

A floating-point comparator returns a <= b for sorting. Which property does this violate?

Follow-up Would adding a tolerance in a < b - eps always fix it?

S05 Answer cues 20 ›

A hot loop repeatedly calls reserve(size()+1) before push_back. How can this defeat vector's amortized growth?

Follow-up Which measurements would expose the regression?

S06 Answer cues 20 ›

A sorted linked list has a million nodes. Is std::lower_bound a logarithmic-time lookup on it?

Follow-up How does a tree container's member lower_bound differ?

S07 Answer cues 20 ›

An unordered_map grows beyond its load-factor threshold. Which cached handles survive rehash?

Follow-up What does erasing one element do to its references?

S08 Answer cues 20 ›

A team needs the ten largest unique scores from a stream. How would you bound memory and handle duplicates?

Follow-up Give a complexity bound as k changes.

QUESTION SET G

Templates and modern C++

Apply deduction and modern library features with a stated language version.

G01 Answer cues 21 ›

How would you write a wrapper that forwards each argument to a callable while preserving its value category?

Follow-up Why is std::move on every argument wrong?

G02 Answer cues 21 ›

A function template is declared in a header and defined in a .cpp. A new argument type links unsuccessfully. Why?

Follow-up When is explicit instantiation a reasonable boundary?

G03 Answer cues 21 ›

A generic algorithm requires random access and sortable elements. How could a C++20 interface state that contract?

Follow-up Which semantic requirements are still the caller's responsibility?

G04 Answer cues 21 ›

A C++17 template has one branch valid only for integral types. When does if constexpr help?

Follow-up Why can an invalid nondependent name still cause an error?

G05 Answer cues 21 ›

A generic function returns auto from an expression yielding T&. How would you preserve the reference intentionally?

Follow-up What lifetime mistake can decltype(auto) expose?

G06 Answer cues 21 ›

Use std::span<const double> for a statistics routine. Which callers can use it, and what does it promise?

Follow-up How would an asynchronous version retain the input?

G07 Answer cues 21 ›

An API currently signals failure with an empty string. When would optional, expected or a variant improve it?

Follow-up Which of those is a C++23 standard-library facility?

G08 Answer cues 21 ›

A range pipeline is stored and iterated after a local source goes out of scope. What would you inspect?

Follow-up What if the predicate captures a local threshold by reference?

QUESTION SET E

Exceptions and API contracts

State an invariant, an error channel and the guarantee on failure.

E01 Answer cues 22 ›

A container operation promises the strong exception guarantee. How would you implement a replacement that may allocate and throw?

Follow-up What must be true about the commit step?

E02 Answer cues 22 ›

A file-writing destructor can encounter a flush error. How do clients find out without throwing during stack unwinding?

Follow-up What work should a destructor still perform?

E03 Answer cues 22 ›

Public code catches std::exception by value and rethrows with throw e. What information can be lost?

Follow-up How do you add context and preserve the original failure?

E04 Answer cues 22 ›

An imported file can be malformed, unavailable or simply contain zero records. Design the parse result.

Follow-up How should callers distinguish a legitimate empty file from a failure?

E05 Answer cues 22 ›

A C++ plug-in is called through a C ABI. An exception might escape a callback. Where would you translate it?

Follow-up How can the host retrieve a diagnostic with a safe lifetime?

E06 Answer cues 22 ›

A class declares move assignment noexcept, but one of its members can throw on assignment. What outcome is observable?

Follow-up How do you derive a correct exception specification?

E07 Answer cues 22 ›

A batch optimizer mutates several model entities then fails on the fourth one. How should the API define recovery?

Follow-up Can the operation be retried safely?

E08 Answer cues 22 ›

An internal assertion checks that a user-supplied index is within bounds. Is that enough in a release build?

Follow-up How would you handle an impossible internal invariant differently?

QUESTION SET T

Concurrency and the memory model

Identify shared state, the synchronization relationship and the shutdown path.

T01 Answer cues 23 ›

Two threads increment the same ordinary int with no lock. What does the language guarantee?

Follow-up Would declaring the counter volatile fix the issue?

T02 Answer cues 23 ›

A worker publishes a completed result through an atomic ready flag. What ordering makes the non-atomic payload visible?

Follow-up When would relaxed ordering be sufficient?

T03 Answer cues 23 ›

A queue consumer uses a condition variable. Why must it wait with a predicate?

Follow-up How should the predicate change when the queue supports close()?

T04 Answer cues 23 ›

Two account transfers lock source first and destination second. What interleaving deadlocks?

Follow-up How do you handle transfer from an account to itself?

T05 Answer cues 23 ›

A worker thread captures this and is detached. The owner can be destroyed during shutdown. How would you redesign it?

Follow-up Can std::jthread interrupt a blocking read by itself?

T06 Answer cues 23 ›

Multiple threads copy shared_ptr instances referring to one vector. May they append concurrently?

Follow-up What if they also assign to the same shared_ptr variable?

T07 Answer cues 23 ›

A callback is invoked while a registry holds its mutex. Which failure modes should worry you?

Follow-up Can you release the lock without calling a destroyed subscriber?

T08 Answer cues 23 ›

A profiler shows different threads contending while updating separate counters. What is false sharing?

Follow-up How would you distinguish it from a data race?

QUESTION SET D

Debugging, performance and builds

Explain how each observation would support or disprove a hypothesis.

D01 Answer cues 24 ›

A release build crashes after a vector grows. What evidence would you collect before editing the code?

Follow-up Which sanitizer would you try first, and what would you preserve?

D02 Answer cues 24 ›

An application freezes with all CPUs mostly idle. How does your investigation differ from investigating high CPU?

Follow-up What evidence would distinguish deadlock from slow I/O?

D03 Answer cues 24 ›

A timing patch reports a 30% speedup on one run. What would you require before calling it a performance improvement?

Follow-up What if throughput improves while p99 latency regresses?

D04 Answer cues 24 ›

A free function changed signature in a public header. One machine reports an unresolved external. How would you isolate it?

Follow-up What if it only fails in the Release configuration?

D05 Answer cues 24 ›

A service's memory grows while a leak checker reports no unreachable blocks. What else can explain growth?

Follow-up What measurement separates live objects from allocator retention?

D06 Answer cues 24 ›

A structure-of-arrays proposal claims a faster vertex update than an array of structures. Under which access patterns might it win?

Follow-up What could make it slower?

D07 Answer cues 24 ›

A plug-in works in a local debug build but crashes when its host frees a returned buffer. How do you test the ownership boundary?

Follow-up How would you design the next interface version?

D08 Answer cues 24 ›

A sanitizer build and a unit test disagree about whether a bug exists. How do you interpret them?

Follow-up Which tools would you choose for an integer overflow and for a race?

QUESTION SET R

Code review at the whiteboard

Assume standard headers and sensible declarations for helper functions. Review the fragments independently.

R01 Answer cues 25 ›

```
std::string_view label() {
    std::string s = "mesh";
    return s;
}
```

What happens to the returned view when label() finishes? Propose a safe return contract.

Follow-up Would returning const std::string& improve it?

R02 Answer cues 25 ›

```
std::vector<int> v{4, 9};
auto* p = &v.front();
v.push_back(16);
use(*p);
```

Under which condition does the final read through p become invalid?

Follow-up How can a test force that condition reliably?

R03 Answer cues 25 ›

```
void consume(std::string s);
std::string t{"mesh"};
consume(std::move(t));
```

After moving t into consume, which operations are generally safe on t?

Follow-up How would this change if consume took const std::string&?

R04 Answer cues 25 ›

```
for (auto it = v.begin(); it != v.end(); ++it) {
    if (*it < 0) v.erase(it);
}
```

Why can this erase loop skip elements or access an invalid iterator?

Follow-up Give a one-pass, order-preserving alternative.

QUESTION SET R

Code review at the whiteboard

Assume standard headers and sensible declarations for helper functions. Review the fragments independently.

R05 Answer cues 25 ›

```
auto makeNext() {
    int n = 0;
    return [&n] { return ++n; };
}
auto next = makeNext();   // caller invokes next twice
```

Is the result of both calls to next() portable and well-defined?

Follow-up What would change if the counter were a local static protected by a mutex?

R06 Answer cues 25 ›

```
struct Base { ~Base() = default; };
struct Derived : Base { std::vector<int> data; };
std::unique_ptr<Base> p = std::make_unique<Derived>();
```

When does this destruction path release the derived resource correctly?

Follow-up How would you make incorrect ownership impossible at the interface?

R07 Answer cues 25 ›

```
// Initially ready is atomic<bool>{false}, payload is int{0}.
// Writer thread
payload = 42; ready.store(true, std::memory_order_relaxed);
// Reader thread
if (ready.load(std::memory_order_relaxed)) use(payload);
```

Is this flag enough to publish payload to the reader?

Follow-up What if payload is written again while the reader uses it?

R08 Answer cues 25 ›

```
std::sort(v.begin(), v.end(), [](int a, int b) {
    return a <= b;
});
```

For this comparator, can sort's ordering requirements hold on all integer inputs?

Follow-up Supply a concrete counterexample and a corrected ordering.

QUESTION SET A

Live coding and algorithms

Clarify the contract, choose an algorithm, walk an edge case and give its cost.

A01 Answer cues 26 ›

Implement firstIndex(sorted, target) for a sorted vector of integers. Return the first matching index or no value.

Follow-up Explain the invariant and test an empty vector and a duplicate at the beginning.

A02 Answer cues 26 ›

Implement an LRU cache with get and put, fixed capacity and replace-on-update. Assume integer keys and values.

Follow-up How should capacity zero and an update to the most recent item behave?

A03 Answer cues 26 ›

Merge overlapping closed integer intervals and return them in ascending order.

Follow-up Should [1,2] merge with [2,3]? What about [3,4] after [1,2]?

A04 Answer cues 26 ›

Remove duplicate IDs from a vector while preserving the order of first occurrences.

Follow-up How would tight memory limits alter the trade-off?

A05 Answer cues 26 ›

Find an order in which build targets can be compiled, or report a dependency cycle.

Follow-up How will a target with no dependencies and a disconnected component be handled?

A06 Answer cues 26 ›

Build a bounded blocking queue with push, pop and close. State your shutdown contract before coding.

Follow-up What do blocked producers and consumers do on close?

A07 Answer cues 26 ›

Return the top k most frequent error IDs, ordering ties by ID.

Follow-up How do you make output deterministic when the input is shuffled?

A08 Answer cues 26 ›

Implement an owning dynamic array of int with deep copy and noexcept move. Support construction, indexing, copy and move.

Follow-up How would you prove independence after copying and safety after self-assignment?

QUESTION SET X

CAD, geometry and plug-ins

Specify units, reference frames, degeneracies and the host ownership boundary.

X01 Answer cues 27 ›

Given a point and two endpoints in 3D, return the closest point on the segment.

Follow-up What if the endpoints coincide or the projection lies beyond an endpoint?

X02 Answer cues 27 ›

Two faces are considered coincident within a tolerance. What can go wrong with one global epsilon?

Follow-up How would you test the rule when moving drawings between metres and millimetres?

X03 Answer cues 27 ›

Describe a 2D convex hull implementation for unordered points, including duplicates.

Follow-up How do you want to handle collinear edge points and overflow in orientation?

X04 Answer cues 27 ›

A face ID cached by a plug-in refers to a different face after a regeneration. How would you design selection persistence?

Follow-up How do you present ambiguity to the caller?

X05 Answer cues 27 ›

A transform is correct at the origin but wrong when a component is rotated in a parent assembly. How would you debug it?

Follow-up Which minimal test distinguishes transform order from a unit mismatch?

X06 Answer cues 27 ›

Computing intersections for every pair of n bounding boxes is slow. How would you reduce candidate pairs?

Follow-up Can the worst case still be quadratic?

X07 Answer cues 27 ›

A mesh import produces zero-area triangles and indexing beyond the vertex array. Design validation and reporting.

Follow-up Which repairs may change geometry and need an explicit policy?

X08 Answer cues 27 ›

A geometry callback comes from the host UI thread, but a solver is slow. How can you move work off-thread?

Follow-up What if the user edits or closes the document while the solver runs?

QUESTION SET W

Delivery and engineering judgment

Use real examples and measurements you could explain to a technical reviewer.

W01 Answer cues 28 ›

Walk through a production defect you owned from first report to verified fix.

Follow-up Which observation ruled out your first hypothesis?

W02 Answer cues 28 ›

An automated build is much faster than the old process. How would you defend the claimed improvement in an interview?

Follow-up What were the starting and ending boundaries for each measurement?

W03 Answer cues 28 ›

A release branch receives two simultaneous build requests. What should the pipeline guarantee?

Follow-up How should a failed upload be retried without publishing the wrong artifact?

W04 Answer cues 28 ›

QA reports a geometry regression that appears only in one customer file. How do you decide what to log and test?

Follow-up How would you create a shareable minimal reproducer?

W05 Answer cues 28 ›

Describe a choice you made between implementing an abstraction and shipping a narrow fix.

Follow-up What future evidence would justify revisiting the decision?

W06 Answer cues 28 ›

A reviewer disagrees with your choice of owning pointer in a shared API. How do you resolve the disagreement?

Follow-up What would convince you to change your proposal?

W07 Answer cues 28 ›

An integration estimate is requested before anyone has validated an external SDK call. How would you present a credible plan?

Follow-up Which finding should trigger a revised estimate?

W08 Answer cues 28 ›

A change passed unit tests but failed in staging. How would you decide whether to roll back or patch forward?

Follow-up What evidence closes the incident after service is restored?

ANSWER CUES / SET L

Lifetime, references and linkage

Compare your answer with these checkpoints. Select a question link to try the prompt again.

L01 Question 4 ›

A view contains a pointer and a length. The backing characters must still exist at the same address. Reallocation, destruction or a temporary argument can break the contract. Return an owning string if the caller cannot bound its use.

L02 Question 4 ›

They bind to T&, const T& and T&& respectively. The named parameter expression is an lvalue. An explicit move can select a consuming overload when the function owns that decision.

L03 Question 4 ›

The copy's pointer still addresses the original storage. A move may also change the address, especially for short strings. Store an offset, or reconstruct the pointer in every special member that changes ownership.

L04 Question 4 ›

Each translation unit compiles an ordinary definition with external linkage. Multiple definitions violate the one-definition rule. Move the definition to one source file or use inline with equivalent definitions wherever the rules allow it.

L05 Question 4 ›

An ordinary non-extern namespace-scope const variable generally has internal linkage. Explicit static also gives internal linkage. An inline constexpr variable with external linkage can appear in several translation units and denotes one entity with one address.

L06 Question 4 ›

Multiplication and division inside the parentheses use integers, so t=1 gives zero. Converting before multiplication, for example 0.5 * 9.8 * t * t, preserves the fraction and avoids signed overflow in the integer square.

L07 Question 4 ›

Initialization by competing threads occurs once on successful completion. If initialization throws, a later call retries. Later mutable access needs its own synchronization or an immutable design.

L08 Question 4 ›

Alignment, object lifetime, type access, width and byte order all matter. Decode unsigned bytes with shifts, or copy into a suitable trivially copyable integer and convert endianness. Check the remaining buffer length before either approach.

ANSWER CUES / SET M

Ownership and move semantics

Compare your answer with these checkpoints. Select a question link to try the prompt again.

M01 Question 5 ›

Store exactly one owning handle and release a valid one once. Delete copy operations, transfer and invalidate on move, and leave the object destructible after self-move. Use a factory or an explicit error path if acquisition can fail.

M02 Question 5 ›

std::move is a cast. For a const string, the usual string move constructor cannot bind to const&&, so a copy can be selected. Other types may implement expensive moves. Inspect the selected operation and its preconditions.

M03 Question 5 ›

Return the local by name to permit NRVO. If NRVO does not occur, an eligible local can be moved. Returning an explicit move of the local normally prevents NRVO. A same-type prvalue constructs the result directly in C++17.

M04 Question 5 ›

A required observer can take a reference and an optional observer a pointer. Use unique_ptr by value to transfer unique ownership. Take shared_ptr by value when the callee needs its own share. Document any lifetime that crosses the call.

M05 Question 5 ›

A vector can copy existing elements to preserve their original values if relocation fails. A genuinely nonthrowing move permits cheap relocation. When a throwing move is the only choice, the usual strong guarantee may be waived for that operation.

M06 Question 5 ›

There are two control blocks and both may delete the same object. Create one owner and copy it. An object already under suitable shared ownership can use enable_shared_from_this. Calling shared_from_this before ownership is established can throw bad_weak_ptr.

M07 Question 5 ›

Decide which relationship truly owns lifetime. Make the back-reference weak or manage both from an enclosing owner. In a callback, call lock once and keep the resulting shared_ptr alive while using the object.

M08 Question 5 ›

A user-declared destructor suppresses implicit move generation. An rvalue may bind to a copy constructor. Prefer the rule of zero, or explicitly declare the required copy and move operations and verify their exception specifications.

ANSWER CUES / SET P

Polymorphism and object design

Compare your answer with these checkpoints. Select a question link to try the prompt again.

P01 Question 6 ›

Ordinary deletion through Base* requires a virtual base destructor here. A protected nonvirtual destructor prevents clients from deleting through the base when an alternative destruction protocol is required.

P02 Question 6 ›

Copying into a base value slices the derived part. Use a reference to observe one shape and a container of owning pointers for a heterogeneous collection. A variant can suit a closed set of alternatives.

P03 Question 6 ›

During base construction, virtual dispatch targets the class currently under construction. Derived state is unavailable. Pass the required data to constructors or have a factory invoke explicit setup after full construction.

P04 Question 6 ›

Mark overrides with override so the compiler checks the signature. A derived declaration can hide other overloads with the same name. A using declaration can reintroduce the base overload set where appropriate.

P05 Question 6 ›

Inject a strategy object or callable into the export service. Define ownership, job-local lifetime and replacement policy explicitly. Composition permits independent tests and avoids imposing an unrelated base relationship on entities.

P06 Question 6 ›

Return unique_ptr<Shape> throughout and construct the concrete copy inside each override. Smart pointer specializations are distinct return types. Language-level covariant returns apply to qualifying raw pointers or references to classes.

P07 Question 6 ›

Compare openness of the type set, binary boundaries, editing cost and data layout. Measure the actual mix of types, allocations and access order. Dispatch cost can be smaller than cache misses or work performed by each operation.

P08 Question 6 ›

A caller that honors the base contract can now fail for a particular dynamic type. Make common preconditions and postconditions explicit. Run the same contract suite through base references for each implementation.

ANSWER CUES / SET S

Containers and algorithms

Compare your answer with these checkpoints. Select a question link to try the prompt again.

S01 Question 7 ›

Reallocation invalidates all element addresses. With sufficient capacity, push_back preserves prior element addresses, although the old end iterator changes. Erasing invalidates iterators and references at or after the erased position.

S02 Question 7 ›

Compare sorted iteration, locality, mutation and latency requirements. A map offers logarithmic lookup in key order. A hash table offers average constant lookup with linear worst case. A sorted vector offers binary search and linear insertion.

S03 Question 7 ›

Use erase-remove_if or C++20 erase_if. The compaction pass is linear and preserves survivor order. If erasing a single element during iteration, use the iterator returned by erase.

S04 Question 7 ›

A strict weak ordering must be irreflexive. The original comparator says equal values precede themselves. A tolerance can make equivalence nontransitive. Specify a consistent ordering or quantize into explicit buckets, and define a NaN policy.

S05 Question 7 ›

Repeated exact reservations can force frequent reallocations and cumulative quadratic relocation. Reserve a credible total once or allow normal growth. Instrument allocation count and moved bytes across increasing input sizes.

S06 Question 7 ›

The generic algorithm uses logarithmically many comparisons but can require linear iterator increments on forward or bidirectional iterators. A balanced tree's member function descends its own structure in logarithmic time.

S07 Question 7 ›

Rehash invalidates iterators. References and pointers to existing elements remain valid through a rehash. Erasing an element invalidates handles to that element. Do not carry iterators across an insert that may rehash.

S08 Question 7 ›

Maintain at most k values in an ordered set, or a heap plus membership set. Discard duplicates and evict the current minimum when warranted. A balanced set gives O(n log(k+1)) time and O(k) storage, with explicit handling for k=0.

ANSWER CUES / SET G

Templates and modern C++

Compare your answer with these checkpoints. Select a question link to try the prompt again.

G01 Question 8 ›

Deduce Args&& and apply std::forward<Args>(args)... at the call site. A forwarding reference can represent an lvalue. Unconditionally moving would let the wrapper consume a caller's lvalue. std::invoke also supports member pointers.

G02 Question 8 ›

Implicit instantiation generally requires the definition to be reachable where needed. Put the definition in the header, or explicitly instantiate the finite supported set in an implementation file. New types then need new instantiations.

G03 Question 8 ›

Use appropriate iterator or range concepts and a sortable constraint. Constraints improve overload selection and diagnostics. They cannot generally prove that a runtime comparator obeys strict weak ordering.

G04 Question 8 ›

The discarded branch is not instantiated for that specialization when the condition is resolved during instantiation. Parsing and nondependent name lookup still apply. Keep type-specific expressions dependent on the template parameters.

G05 Question 8 ›

Plain auto by value drops the reference. An explicit reference return or carefully chosen decltype(auto) can preserve it. Parenthesizing a named local in a decltype(auto) return can deduce a dangling reference.

G06 Question 8 ›

A span describes contiguous elements and their count. It borrows storage and does not extend lifetime. Read-only element access does not make other aliases immutable. For deferred work, use an owning snapshot or a documented shared lifetime.

G07 Question 8 ›

optional represents presence without a reason. expected<T,E> distinguishes a value from a structured error and is standard in C++23. A variant models several meaningful alternatives. Define whether an empty string is valid data.

G08 Question 8 ›

Determine which views own the underlying range and which borrow it. Laziness defers iteration and predicate calls. Ensure both source and captured state outlive use, capture by value where suitable, or materialize the results.

ANSWER CUES / SET E

Exceptions and API contracts

Compare your answer with these checkpoints. Select a question link to try the prompt again.

E01 Question 9 ›

Build the replacement in temporary storage, validate it, then commit with an operation that cannot fail under the relevant allocator and type rules. If preparation fails, the old observable value remains intact.

E02 Question 9 ›

Offer an explicit close or commit operation that reports failures while recovery is possible. Keep destructor cleanup nonthrowing. In unwinding, an escaping second exception can terminate the process.

E03 Question 9 ›

Catch by const reference and use bare throw to rethrow the current exception. Catching or throwing as the base can slice. Log context once or deliberately use a nested exception or structured error.

E04 Question 9 ›

Return a successful empty collection for a valid empty file. Represent failures separately with a category, source location where relevant and diagnostic detail. Specify how partial records, cancellation and I/O failures are handled.

E05 Question 9 ›

Catch at the boundary and translate into the ABI's error protocol. Define buffer ownership or use a caller-supplied buffer. Keep allocation, deallocation and exception representation inside compatible modules.

E06 Question 9 ›

If an exception escapes a noexcept operation, termination follows. Prefer defaulted special members when appropriate or express conditional noexcept from the operations actually used. Test the throwing path and document invariants.

E07 Question 9 ›

Choose an atomic transaction or an explicitly partial result with per-entity status. Stage changes or journal reversible edits, including side effects. Define stable operation IDs or detect prior application if retry is possible.

E08 Question 9 ›

Validate externally supplied values on the production path and return an error, throw or reject according to the contract. An assertion is useful for internal assumptions but can be compiled out. Exercise boundary values in release-like tests.

ANSWER CUES / SET T

Concurrency and the memory model

Compare your answer with these checkpoints. Select a question link to try the prompt again.

T01 Question 10 ›

Concurrent conflicting non-atomic accesses without a happens-before relation constitute a data race and yield undefined behavior. Use a mutex or an atomic increment. Volatile does not provide inter-thread synchronization.

T02 Question 10 ›

Write the payload before a release store. Read it after an acquire load that observes that publication, with no later conflicting writes. Relaxed ordering can suffice for an independent statistic that publishes no other data.

T03 Question 10 ›

A wakeup can be spurious or another consumer can take the item first. Under the same mutex, test whether work exists or the queue is closed. A closed and empty queue tells consumers to exit. Notify waiters when closing.

T04 Question 10 ›

Opposite-direction transfers can each hold the other's required mutex. Use a common lock order or std::scoped_lock on distinct mutexes. Handle identical accounts before attempting to lock the same nonrecursive mutex twice.

T05 Question 10 ›

Tie the thread to an owner, request cooperative stop and unblock pending work before joining. Ensure the state used by the worker survives until the join completes. A stop request alone cannot cancel arbitrary blocking I/O.

T06 Question 10 ›

The control block supports concurrent ownership operations through distinct shared_ptr objects. The pointee needs its own synchronization. Concurrent writes to one shared_ptr object also require synchronization, such as C++20 atomic<shared_ptr<T>>.

T07 Question 10 ›

Callbacks can reenter the registry, block or take locks in a conflicting order. Select or snapshot live callback handles under the lock. Invoke after release and define whether an already-selected callback can run after unsubscribe.

T08 Question 10 ›

Independent writable objects can occupy one cache line, causing coherence traffic without a language-level race. Compare per-thread accumulation, aligned or padded layouts and hardware counters under the same load. A race detector addresses a different failure mode.

ANSWER CUES / SET D

Debugging, performance and builds

Compare your answer with these checkpoints. Select a question link to try the prompt again.

D01 Question 11 ›

Capture the exact build, symbols, input and stack. Reproduce reallocation with a small test and use AddressSanitizer to look for stale accesses. Preserve both allocation and free stacks. A clean run only covers the paths exercised.

D02 Question 11 ›

Capture stacks for all threads more than once, then identify waits, lock owners and pending I/O. A persistent lock cycle suggests deadlock. For hot CPU, sample active stacks and measure where execution time accumulates.

D03 Question 11 ›

Reproduce with identical release flags, representative data, warm-up and repeated measurements. Consume results to prevent dead-code elimination and record variability. Report throughput and tail latency separately against the actual requirement.

D04 Question 11 ›

Compare the declaration, definition, namespace and calling convention. Inspect the actual link command, object symbols, library versions and architecture. Verify the source joins that target and the failing configuration uses the expected binary.

D05 Question 11 ›

Caches, queues and retained capacity can keep memory reachable. Compare object counts and allocation stacks over time with RSS and allocator statistics. Drive a controlled growth and eviction cycle to see what becomes reclaimable.

D06 Question 11 ›

Processing one coordinate or property across many vertices can improve locality and vectorization. Touching every field per vertex may favor an interleaved layout. Measure representative strides, cache behavior, conversion and boundary costs.

D07 Question 11 ›

Inspect allocator and runtime compatibility, object layout, ownership and build flags. Trace the exact allocator/free pair. Export paired create/destroy functions or use caller-provided storage and a versioned interface.

D08 Question 11 ›

A passing test only establishes its observed result on that execution. Investigate sanitizer reports using a minimized reproducer. UBSan can catch signed overflow and TSan can detect races. Build separate supported configurations and inspect each report.

ANSWER CUES / SET R

Code review at the whiteboard

Compare your answer with these checkpoints. Select a question link to try the prompt again.

R01 Question 12 ›

The local string is destroyed at return, leaving the view dangling. Returning a reference to the local has the same lifetime problem. Return the string by value, or take caller-owned storage and document how long it must remain valid.

R02 Question 12 ›

A reallocation invalidates p. Fill the vector to its current capacity, cache an address, then append once more. Reacquire the element after mutation or choose a contract that guarantees stable storage.

R03 Question 12 ›

For a moved-from standard string, destruction, assignment and operations whose preconditions are met are valid. Its contents are unspecified. Merely binding the reference parameter would not move from t.

R04 Question 12 ›

After erase, the original iterator is invalid. Incrementing it is invalid too. Assign the returned iterator when erasing, otherwise increment. For vector, erase-remove_if or erase_if avoids repeated shifts.

R05 Question 13 ›

Each invocation binds a reference to the local n captured by the returned closure. Its lifetime ended when makeNext returned, so invoking it has undefined behavior. Capture n by value in a mutable lambda for independent counters. A protected static would share state across callers.

R06 Question 13 ›

With the shown nonvirtual destructor, deletion via Base* of a Derived object has undefined behavior. A public virtual base destructor makes this polymorphic default-deleter path valid. A factory can instead enforce a custom destruction protocol.

R07 Question 13 ›

Relaxed access to the flag alone provides no release/acquire synchronization for payload. Use release and a matching acquire, or protect the data with a mutex. Subsequent concurrent writes also need their own protocol.

R08 Question 13 ›

With <=, comp(3,3) is true, violating irreflexivity. Use < for ascending order. Include equal keys in tests, and use stable_sort if the relative order of equal records is part of the contract.

ANSWER CUES / SET A

Live coding and algorithms

Compare your answer with these checkpoints. Select a question link to try the prompt again.

A01 Question 14 ›

Maintain a half-open lower-bound interval and converge on the first element at least equal to the target. Check the end and equality before returning. Time O(log n), auxiliary space O(1). Compute the midpoint without adding two large indices.

A02 Question 14 ›

Store recency in a list and map keys to list iterators. Splice to the front on access and evict from the back. Average O(1) per operation, O(capacity) storage, with hash worst cases. Keep map and list consistent if insertion throws.

A03 Question 14 ›

Define valid endpoints, sort by start, then extend the current end on overlap. Closed intervals sharing an endpoint overlap. Separate intervals with a gap stay separate. Time O(n log n) plus output storage O(n).

A04 Question 14 ›

Track seen IDs and emit or compact only the first occurrence. With hashing, average O(n) time and O(u) auxiliary storage for u distinct IDs. Scanning the compacted prefix avoids the set but can take quadratic time.

A05 Question 14 ›

Add directed edges from prerequisite to dependent, including isolated nodes. Use indegrees and a queue, removing newly ready nodes. Fewer than V emitted nodes signals a cycle. Time and storage O(V+E) with adjacency lists.

A06 Question 14 ›

Use one mutex, separate wait predicates for data and space, and a closed flag. Wake both groups on close, reject new pushes and drain queued items before pop returns closed. Test with full and empty queues and multiple waiters.

A07 Question 14 ›

Count occurrences, then select with a size-k heap ordered by frequency and the specified ID tie-break. Sort the final k using the required output ordering. For u IDs, average O(n + u log(k+1) + k log(k+1)) time and O(u+k) storage.

A08 Question 14 ›

Manage allocation through RAII. Prepare a copy before replacing existing storage, and leave moved-from size and pointer consistent. Copy costs O(n) time and new storage. Move construction is O(1). Test empty, copy, move and self-assignment explicitly.

ANSWER CUES / SET X

CAD, geometry and plug-ins

Compare your answer with these checkpoints. Select a question link to try the prompt again.

X01 Question 15 ›

Compute t = dot(P-A,B-A) / dot(B-A,B-A), clamp t to [0,1], then return A+t(B-A). Handle the zero-length segment before division. Test endpoints, an interior projection and a point beyond each end.

X02 Question 15 ›

Define units and whether the tolerance refers to distance, angle or relative scale. Use domain-appropriate tolerances and record the coordinate frame. Test near the decision boundary at two scales and specify how unit conversion scales length tolerances.

X03 Question 15 ›

Sort and deduplicate, build lower and upper chains with an orientation test and combine. State whether to retain boundary-collinear points. Widen before subtraction and multiplication when bounds permit, or use robust predicates. Time O(n log n).

X04 Question 15 ›

Prefer a host-supported persistent reference when available. Track model identity and revision, validate resolved geometry and surface ambiguous matches explicitly. Test topology changes, save/reopen and model copies separately.

X05 Question 15 ›

State source and destination frames and matrix convention. Test identity, pure translation, pure rotation and a noncommuting combination on the origin and basis vectors. Verify units and parent-child multiplication order.

X06 Question 15 ›

Choose a spatial index or sweep based on update frequency, dimension and distribution. Use a broad phase to generate candidates and verify them in a narrow phase. If all boxes overlap, the output alone can contain $\Theta(n^2)$ pairs.

X07 Question 15 ›

Check index ranges before dereference, reject nonfinite coordinates and classify repeated vertices or near-zero area with stated units and tolerances. Separate diagnostics from optional repairs such as welding or dropping faces. Retain source element IDs for reports.

X08 Question 15 ›

Snapshot safe data on the allowed thread, tag the model revision, compute off-thread and marshal results back through the host API. Check document identity, revision and cancellation before committing results.

ANSWER CUES / SET W

Delivery and engineering judgment

Compare your answer with these checkpoints. Select a question link to try the prompt again.

W01 Question 16 ›

Give the reproduction, specific evidence, root cause, smallest safe fix and regression test. Separate your own contribution from team actions. Explain how you verified the fix in the environment where it failed.

W02 Question 16 ›

Describe the old and new triggering conditions, representative samples, hardware, cache state, queue time and failure rate. Show the arithmetic with actual measurements. Separate elapsed build time from engineer waiting time.

W03 Question 16 ›

Pin revisions and inputs, isolate working directories and identify artifacts uniquely. Serialize conflicting publication or use an atomic promotion protocol. Retry with the same verified artifact and report each request's outcome.

W04 Question 16 ›

Capture version, units, configuration and relevant entity relationships. Reduce the file while preserving the failure and protecting customer content. Add a targeted geometric invariant or tolerance-based regression test.

W05 Question 16 ›

Describe the requirements, alternatives, likely callers and cost of change. Explain which behavior you kept stable and how it was tested. Identify a concrete trigger for revisiting the abstraction.

W06 Question 16 ›

Draw the ownership and destruction paths, including callbacks and failures. Evaluate alternative signatures against the actual call sites. Use a small prototype or test to settle disputed lifetime assumptions and record the decision.

W07 Question 16 ›

Separate known implementation work from the uncertain dependency. Time-box a spike with a concrete acceptance test, then give a range and milestones. Re-estimate when the API or thread-affinity assumptions are tested.

W08 Question 16 ›

Assess impact, rollback safety, data compatibility and confidence in a fix. Preserve diagnostics and agree on a recovery decision with the owner. Close with a verified restoration, root cause, regression coverage and follow-up ownership.

FURTHER READING

Technical references

The questions and answer cues were written for this bank. These pages support deeper review.

C++ Core Guidelines

Ownership, contracts, interfaces and concurrency

isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines

Object lifetime and the one-definition rule

L and R · object lifetime, storage and definitions

eel.is/c++draft/basic.life
eel.is/c++draft/basic.def.odr

Copy elision and delete expressions

M and P · construction of results and polymorphic deletion

eel.is/c++draft/class.copy.elision
eel.is/c++draft/expr.delete

Containers and sorting

S and A · complexity, iterator requirements and ordering

eel.is/c++draft/container.requirements
eel.is/c++draft/alg.sorting

Deduction and exception specifications

G and E · forwarding references and noexcept

eel.is/c++draft/temp.deduct.call
eel.is/c++draft/except.spec

C++ concurrency and condition variables

T · synchronization and predicate waits

eel.is/c++draft/thread
eel.is/c++draft/thread.condition.condvar

cppreference language and library reference

S, G and T · version-aware entries for vector, span and data races

en.cppreference.com/w/cpp/container/vector.html
en.cppreference.com/w/cpp/container/span.html
en.cppreference.com/w/cpp/language/multithread.html

Clang instrumentation guides

D and R · memory misuse, undefined behavior and data races

clang.llvm.org/docs/AddressSanitizer.html
clang.llvm.org/docs/UndefinedBehaviorSanitizer.html
clang.llvm.org/docs/ThreadSanitizer.html